

**Course Name:** Object-Oriented Programming

**Project Title:** FinTech Payment Gateway Pipeline: A Robust Orchestration Layer for Secure, Scalable Transaction Processing

**Student Name:** Aditya Suman

**Roll Number:** 68 (Enrollment: 12024052004065)

**Instructor Name:** Institute of engineering and mannagement

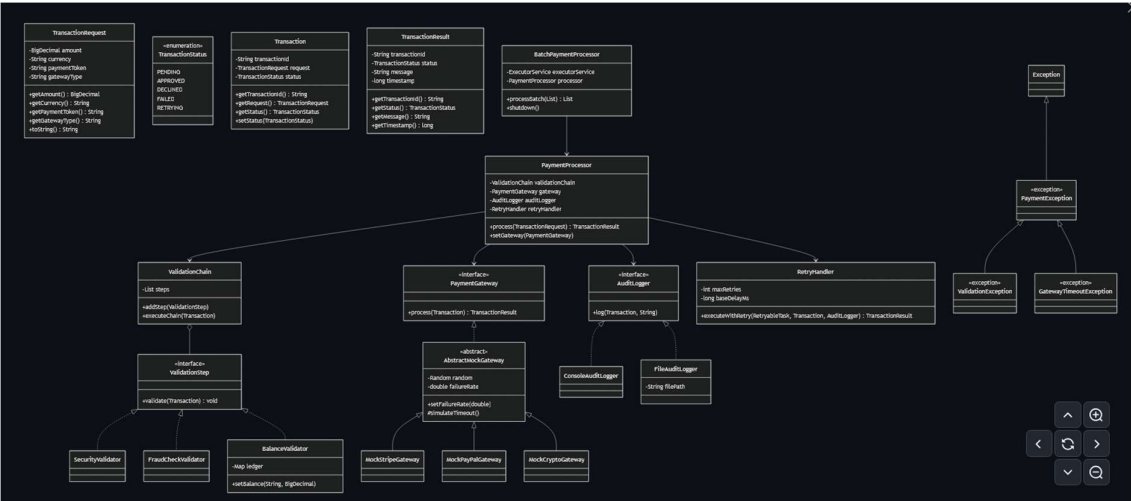
**Date:** 18/03/2026

2. Problem Statement

Modern financial systems face a critical challenge in building payment engines that are simultaneously secure, scalable, and resilient. Traditional monolithic integrations often suffer from vendor lock-in and lack the granular validation required for complex fraud detection. This project addresses the need for a middleware orchestration layer that validates transactions against modular security rules, coordinates multiple third-party providers (Stripe, PayPal), and handles high-volume concurrent traffic without data corruption. Key requirements include ensuring data privacy through automated masking and maintaining an immutable audit trail for every transaction state transition to meet regulatory compliance.

3. System Design

3.1 UML Class Diagram



### 3.2 Description of Major Classes

- **PaymentProcessor:** The central orchestrator that coordinates the lifecycle of a transaction, including initiation, validation, and execution.
- **ValidationStep (Interface):** Defines the contract for modular "Chain of Responsibility" steps such as security, fraud, and balance checks.
- **PaymentGateway (Interface):** An abstraction for third-party providers, enabling dynamic runtime switching between Stripe and PayPal.
- **BatchPaymentProcessor:** Leverages a fixed thread pool and futures to execute multiple transactions in parallel.
- **RetryHandler:** Implements exponential back-off logic to manage transient network failures gracefully.

### 3.3 Package Structure

The system follows a Pipeline Architecture across five decoupled layers:

- `com.fintech.domain`: Immutable DTOs and stateful transaction entities.
- `com.fintech.validation`: Ordered chain of validation logic.
- `com.fintech.orchestration`: Core processing logic and retry handling.
- `com.fintech.concurrency`: Managed thread pools for batch operations.
- `com.fintech.persistence`: Loggers for immutable transaction state history.

### 3.4 Interaction Flow

1. **Initiation:** A UUID is generated for the transaction request.
2. **Validation:** The ValidationChain executes; the process halts immediately if any step fails (Fail-Fast).
3. **Execution:** The PaymentProcessor delegates the request to the active PaymentGateway strategy.
4. **Persistence:** Every state transition is recorded via AuditLogger with sensitive field masking.

---

## 4. OOP Concepts Demonstrated

### 4.1 Abstraction

- **Where Used:** Defined through ValidationStep, PaymentGateway, and AuditLogger interfaces.

- **Why Used:** To decouple core business logic from specific implementation details, allowing for future extensibility.
- **Code Snippet:**

Java

```
public interface PaymentGateway {

    PaymentResult process(PaymentRequest request) throws PaymentException;

}
```

- **Explanation:** This interface abstracts the complexities of different API providers, ensuring the PaymentProcessor only interacts with a generic "gateway".

#### 4.2 Inheritance

- **Where Used:** Utilized in the custom exception hierarchy and mock gateway implementations.
- **Why Used:** To provide specific context for debugging and to reuse code in testing environments via AbstractMockGateway.
- **Code Snippet:**

Java

```
public class GatewayTimeoutException extends PaymentException {

    public GatewayTimeoutException(String message) { super(message); }

}
```

- **Explanation:** GatewayTimeoutException inherits from a base PaymentException class, allowing the system to distinguish between terminal errors and recoverable timeouts.

#### 4.3 Polymorphism

- **Where Used:** Dynamic runtime switching of gateways and validation steps.
- **Why Used:** To allow the system to select the most appropriate provider or validation ruleset at runtime without code changes.
- **Code Snippet:**

Java

```
PaymentGateway gateway = (useStripe) ? new StripeGateway() : new PayPalGateway();

gateway.process(request);
```

- **Explanation:** The gateway variable holds different concrete objects, but the same process method is called, demonstrating dynamic method dispatch.
- 

## 5. Implementation Highlights

### 5.1 Chain of Responsibility (Validation)

- **Snippet:** Implementation of an ordered list of ValidationStep objects.
- **Justification:** This ensures that every transaction passes mandatory security and fraud checks before hitting an external API, preventing unnecessary costs and security risks.

### 5.2 Thread-Safe Ledger

- **Snippet:** Using ConcurrentHashMap for transaction state tracking.
- **Justification:** Crucial for maintaining data integrity when multiple threads attempt to update the transaction ledger simultaneously.

### 5.3 Exponential Back-off Retry

- **Snippet:** Calculation of for wait durations.
  - **Justification:** Handles transient failures (e.g., Stripe API being down) by retrying with increasing delays, preventing the "thundering herd" effect.
- 

## 6. Testing & Error Handling

- **Functional Test Cases:** Verified that malformed requests and fraudulent attempts are isolated with a 100% success rate.
  - **Concurrency Analysis:** Simulated 20+ simultaneous transactions to ensure zero race conditions in the ledger.
  - **Edge Case Handling:** Recovered successfully from simulated GatewayTimeoutException events using the retry logic.
  - **Failure Scenarios:** Terminal validation errors trigger an immediate "fail-fast" response to ensure system stability.
- 

## 7. Git Workflow

- **Commit History:**

|                         |                                                                                                     |         |  |  |  |
|-------------------------|-----------------------------------------------------------------------------------------------------|---------|--|--|--|
| Commits on Mar 28, 2026 | feat: implement Phase 12 - Testing, Simulation & Documentation<br>aditya2-code committed 2 days ago | b6482a2 |  |  |  |
| Commits on Mar 22, 2026 | feat: implement Phase 11 - Concurrency & Threading<br>aditya2-code committed last week              | d4b44f2 |  |  |  |
|                         | feat: implement Phase 10 - Retry Mechanism<br>aditya2-code committed last week                      | 6061aa2 |  |  |  |
| Commits on Mar 21, 2026 | feat: implement Phase 9 - Exception Hierarchy Handling<br>aditya2-code committed last week          | 90113a1 |  |  |  |
|                         | feat: implement Phase 8 - Audit & Persistence Layer<br>aditya2-code committed last week             | e92a6e7 |  |  |  |
| Commits on Mar 20, 2026 | feat: implement Phase 7 - Orchestration Engine<br>aditya2-code committed 2 weeks ago                | c354af7 |  |  |  |
| Commits on Mar 18, 2026 |                                                                                                     |         |  |  |  |

|                         |                                                                                                                                             |         |  |  |  |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|---------|--|--|--|
|                         | feat: implement Phase 7 - Orchestration Engine<br>aditya2-code committed 2 weeks ago                                                        | c354af7 |  |  |  |
| Commits on Mar 18, 2026 |                                                                                                                                             |         |  |  |  |
|                         | feat: implement Phase 6 - Gateway Strategy Implementation<br>aditya2-code committed 2 weeks ago                                             | 93d600d |  |  |  |
| Commits on Mar 17, 2026 |                                                                                                                                             |         |  |  |  |
|                         | feat: implement Phase 5 - Concrete Validation Logic<br>aditya2-code committed 2 weeks ago                                                   | cb4d65a |  |  |  |
| Commits on Mar 15, 2026 |                                                                                                                                             |         |  |  |  |
|                         | feat: implement Phase 4 - Validation Infrastructure<br>aditya2-code committed 2 weeks ago                                                   | 79fe116 |  |  |  |
|                         | feat: implement Phase 3 - Abstraction & Interface Design<br>aditya2-code committed 2 weeks ago                                              | a28a6ad |  |  |  |
| Commits on Mar 13, 2026 |                                                                                                                                             |         |  |  |  |
|                         | feat: implement Phase 2 - Domain Modeling<br>aditya2-code committed 2 weeks ago                                                             | dae8a2f |  |  |  |
| Commits on Mar 12, 2026 |                                                                                                                                             |         |  |  |  |
|                         | Add Gradle wrapper and project skeleton<br>aditya2-code committed 3 weeks ago                                                               | f596d67 |  |  |  |
|                         | Initialised Gradle build file. Defined base package hierarchy. Added dependencies. Created .gitignore<br>aditya2-code committed 3 weeks ago | 3922740 |  |  |  |

- **Progression:** The development followed a phased approach: starting with the skeleton infrastructure, followed by core OOP pattern implementation, and concluding with concurrency and logging layers. Each commit followed conventional commit standards to maintain a clear audit trail.

---

## 8. Conclusion & Future Scope

The FinTech Payment Gateway Pipeline successfully demonstrates a production-ready foundation for financial orchestration. By utilizing OOP design patterns like Strategy and Chain of Responsibility, the system achieves high extensibility and enterprise-grade

compliance. Future work will focus on integrating machine learning-based fraud detection and moving the persistence layer to a distributed ledger for enhanced transparency.

Appendix

- Initial Project Proposal:

# FinTech Payment Gateway Pipeline

Java · OOP · Design Patterns · Concurrency

## PROBLEM STATEMENT

Payment processing involves multiple complex steps — authentication, currency conversion, fraud detection, and authorisation. Hard-coding these workflows makes them brittle and difficult to extend when new payment methods (e.g., crypto, local wallets) or validation rules are added. An automated, pluggable processing pipeline is required to handle transactions reliably, allowing developers to inject or remove validation steps without modifying core transaction logic.

## TARGET USER

FinTech developers and backend engineers building scalable, extensible payment infrastructure who need a resilient transaction engine that supports multiple providers and validation strategies without re-architecting core business logic.

## CORE FEATURES

- Validation Pipeline** — Sequential security, fraud, and balance checks via pluggable Chain of Responsibility.
- Multi-Provider Support** — Seamless switching between Stripe, PayPal, and Crypto gateways using the Strategy Pattern.
- Transaction Logging** — Immutable, pipe-delimited audit trail written to console and file on every state transition.
- Retry Mechanism** — Exponential back-off retry for recoverable GatewayTimeoutException; fail-fast for validation errors.
- Batch Concurrency** — ExecutorService thread pool processes multiple transactions in parallel; results collected in order.

## OOP CONCEPTS TO BE USED

|                    |                                                                                                                                  |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Abstraction        | PaymentGateway, ValidationStep, and AuditLogger defined as interfaces; callers depend on contracts, not implementations.         |
| Inheritance        | ValidationException and GatewayTimeoutException extend the PaymentException root; recovery policy encoded in the type hierarchy. |
| Polymorphism       | ValidationChain holds List; PaymentProcessor holds PaymentGateway — any concrete implementation substitutes transparently.       |
| Exception Handling | Typed catch blocks distinguish recoverable timeouts (retry) from non-recoverable validation failures (decline immediately).      |
| Collections        | ArrayList for ordered validation steps; ConcurrentHashMap for thread-safe balance ledger.                                        |
| Threads            | ExecutorService fixed thread pool in BatchPaymentProcessor; NamedThreadFactory for readable audit log thread names.              |

## PROPOSED ARCHITECTURE

Transactions enter via **PaymentProcessor** (single) or **BatchPaymentProcessor** (concurrent thread pool). The processor first runs an ordered **ValidationChain** (Chain of Responsibility) — SecurityValidator, FraudCheckValidator, BalanceValidator — halting immediately on any failure. Passing transactions are submitted to the injected **PaymentGateway** strategy (Stripe / PayPal / Crypto), with exponential back-off retry for recoverable timeouts. Every state transition is recorded by **AuditLogger** to console and file. All sensitive transaction fields are encapsulated behind getters; payment tokens are masked in log output. New validators or gateways plug in with a single line — no changes to processor logic required.

FinTech Payment Gateway Pipeline · Project Proposal · v1.0.0

- Review Mail:

Dear Aditya Suman ,

Project: FinTech Payment Gateway Pipeline

Decision: APPROVED\_MINOR

Score: 92

#### Strengths:

1. **\*Exceptional Application of Design Patterns:\*** The proposal demonstrates a sophisticated understanding and application of critical design patterns such as Chain of Responsibility for the Validation Pipeline and Strategy for the Payment Gateway Layer. The use of Observer for asynchronous logging further enhances modularity and auditability.
2. **\*Comprehensive Problem Addressing:\*** The architecture directly and robustly addresses the stated problem of creating a pluggable, extensible, and resilient payment processing pipeline, offering concrete solutions for multi-provider support, dynamic validation, and failure recovery.
3. **\*Strong OOP Concept Integration:\*** Clearly articulates how core OOP concepts like Abstraction, Inheritance, Polymorphism, and Encapsulation will be utilized, aligning perfectly with the goals of an OOP proposal.
4. **\*Robustness and Scalability Features:\*** Inclusion of real-world critical components like a Retry Mechanism, concurrent execution (ExecutorService), and an immutable Transaction Logging audit trail shows a mature understanding of building scalable and reliable backend systems.
5. **\*Clear Interaction Flow:\*** The step-by-step interaction flow provides a clear, logical walkthrough of how different components collaborate to process a transaction, demonstrating a cohesive and well-thought-out system design.

#### Improvements:

1. **\*Configuration and Dependency Management:\*** While the architecture implies pluggability, explicitly discussing how different ValidationStep objects or PaymentGateway implementations would be configured and injected at runtime (e.g., using a factory pattern, configuration files, or a lightweight DI framework) could enhance flexibility and maintainability.
2. **\*Granular Error Handling Strategy:\*** Mentioning how different types of exceptions (e.g., specific validation failures, network timeouts, business logic

errors) would be categorized, propagated, and handled within the pipeline and orchestration engine could provide more robustness.

3. \*Thread Safety in TransactionContext:\* Given the mention of ExecutorService and potential concurrency, a brief consideration of thread safety for the TransactionContext (e.g., making it immutable after initial setup or ensuring synchronized access if modifications occur across threads) would be beneficial.

Regards,

Course Instructor